

PicoRV32 FPGA UART Loader Sistemi

Teknik Doküman & Modül Anatomisi

Bellek Haritalandırması · Bellek Konumu Değiştirme
Tüm Modüllerin Detaylı Analizi

Proje	BIL302 Sistem Programlama — Proje 3
Hedef Donanım	Sipeed Tang Nano 9K (Gowin GW1NR-LV9)
İşlemci Çekirdeği	PicoRV32 (RV32I, ENABLE_COUNTERS=0)
Toolchain	Özgün Assembler + Linker (C)
Host Yazılımı	Python + pyserial (XMODEM-CRC16)
Geliştirme Tarihi	Haziran 2026

İçindekiler

#	Bölüm	Sayfa
1	Sistem Genel Bakışı	3
2	Veri Akış Mimarisi	4
3	BELLEK HARİTALANDIRMASI	6
3.1	32-bit Adres Uzayı	6
3.2	RAM Bölgesi — BRAM 32 KB	7
3.3	GPIO Bölgesi	9
3.4	Bayt → Kelime Adresleme	10
3.5	Segment Yerleşimi (.text/.data/stack)	11
4	BELLEK KONUMU DEĞİŞTİRME	13
4.1	5 Katmanlı Sözleşme	13
4.2	Yöntem A: Tüm Mimariyi Taşı	15
4.3	Yöntem B: Trampolin Atlama	16
5	Modül 1 — Assembler	18
6	Modül 2 — Linker	21
7	Modül 3 — Host Loader (Python)	23
8	Modül 4 — UART RX/TX	26
9	Modül 5 — CRC-16	28
10	Modül 6 — Loader FSM	30
11	Modül 7 — memory.v	32
12	Modül 8 — PicoRV32	34
13	Modül 9 — top.v	35
14	Modül 10 — PicoRV32 IDE	36
15	Performans Metrikleri	38
16	Sonuç ve Kaynakça	39

1. Sistem Genel Bakışı

PicoRV32 FPGA UART Loader, kullanıcının yazdığı RISC-V assembly programlarını uçtan uca derleyip Sipeed Tang Nano 9K FPGA üzerinde fiziksel olarak çalıştıran bir **tam yığın (full-stack) gömülü sistem**'dir. Sistem üç ana katmandan oluşur:

1. Bilgisayar (PC) Katmanı: Assembly kaynağını işleyen toolchain (assembler + linker), UART/XMODEM-CRC16 protokolüyle FPGA'e veri gönderen Python host loader ve tüm bunları orkestre eden Tkinter tabanlı görsel IDE.

2. İletişim Katmanı: 115200 baud 8N1 UART üzerinden XMODEM-CRC protokolüyle 128 baytlık paketler hâlinde firmware transferi. CRC-16/XMODEM (polinom 0x1021) ile paket başına %100 veri bütünlüğü.

3. FPGA (Donanım) Katmanı: PicoRV32 RV32I işlemci çekirdeği, 32 KB BRAM, UART alıcı/verici modülleri, donanımsal CRC-16 LFSR hesaplayıcısı, 12-durumlu XMODEM Loader FSM ve memory-mapped I/O (LED + Buton) bloğu.

1.1 Tasarım Hedefleri

- Hızlı geliştirme döngüsü: derleme + yükleme < 1 saniye
- Veri bütünlüğü: paket başına %100 doğrulama (CRC-16)
- Açık kaynak: tüm bileşenler özgün veya BSD/ISC lisanslı (PicoRV32)
- Modüler tasarım: her bileşen bağımsız test edilebilir
- RISC-V psABI uyumu: a0..a7, t0..t6, sp, ra gibi standart ABI isimleri
- Donanım-yazılım birlikte tasarımı (Co-Design): sözleşme tabanlı arayüzler

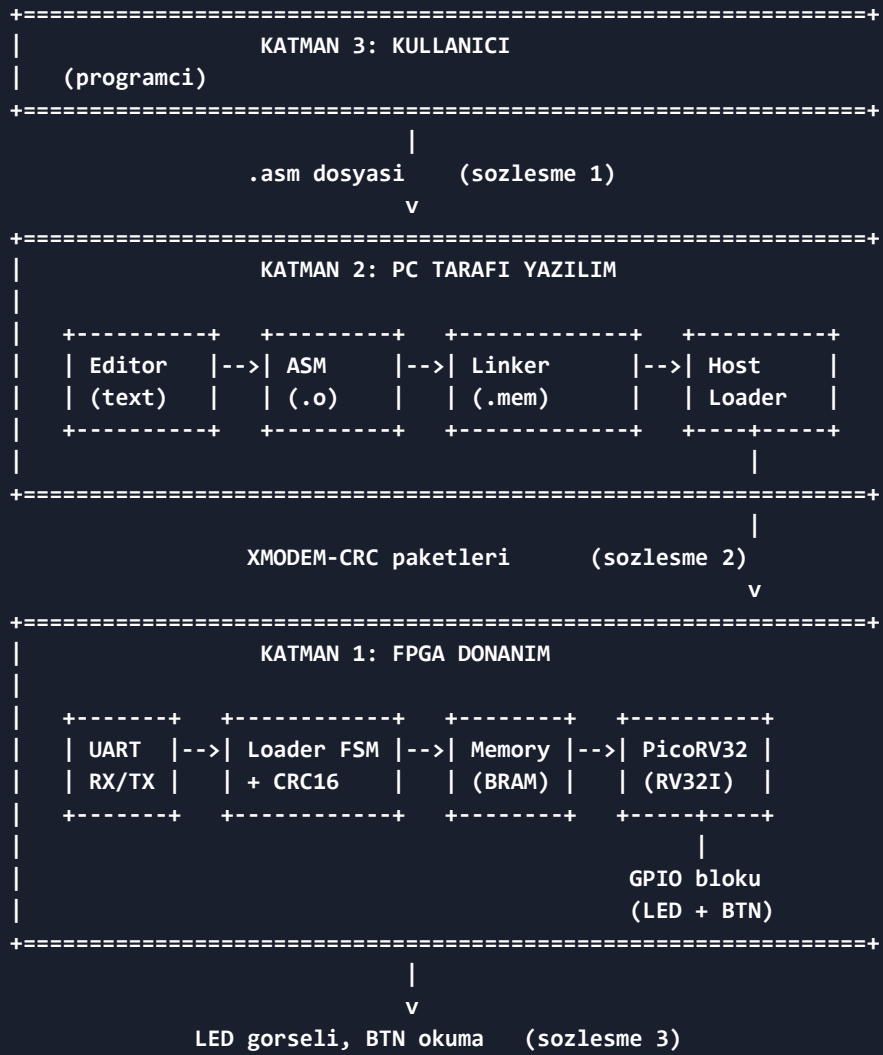
1.2 Sayısal Özet

Bileşen	Değer	Birim
.asm → .o derleme süresi	< 10	ms
.o → .mem linkleme süresi	< 5	ms
UART veri hızı (etkin)	9750	bayt/s
UART teorik maks.	11520	bayt/s
Maks. veri yolu hızı verimi	84.6	%
BRAM kapasitesi	32 768	bayt
XMODEM paket boyutu	128	bayt veri + 4 başlık
CRC-16 polinomu	0x1021	GF(2)
Sistem saati	27	MHz
UART baud rate	115 200	bit/s
FPGA hedef cihazı	GW1NR-LV9	QN88PC6/I5
LUT kapasitesi	8 640	(GW1NR-9C)

Bileşen	Değer	Birim
BSRAM blok sayısı	26	× 16 Kbit
FPGA giriş/çıkış pinleri	63	kullanıcı pin
LED bankı	6	aktif düşük
Buton sayısı	2	S1 reset, S2 kullanıcı

1.3 Sistemin Soyut Mimarisi

Sistem üç ayrı katmandan oluşur ve her katman bir önceki ile sözleşme tabanlı bir arayüz üzerinden konuşur. Bu arayüzler değişmediği sürece her katman içeriği bağımsız geliştirilebilir; bu yaklaşım embedded sistem tasarımında **katmanlı soyutlama** (layered abstraction) prensibini uygular.



Sozlesme 1: RV32I assembly + .asm sentaks

Sozlesme 2: XMODEM-CRC paket formati, 128B + CRC-16/0x1021

Sozlesme 3: Memory-mapped I/O (0x1000_0000), GPIO bitleri

2. Veri Akış Mimarisi

Bir .asm dosyasının fiziksel LED ışıklarına dönüşmesi, sistemin **17 farklı durağı** üzerinden geçen bir yolculuktur. Bu yolculuk PC'de başlar, USB kablosundan geçer, FPGA'in fiziksel pinlerine ulaşır ve sonunda CPU'nun BRAM'den fetch ettiği ilk komut olarak hayata geçer.

2.1 Akış Diyagramı

```

+===== BILGISAYAR (PC) =====+
|
| 1) .asm dosyasi
|   |
|   v
| 2) Assembler -> .o (ham makine kodu + sembol tablosu)
|   |
|   v
| 3) Linker     -> .mem (Verilog $readmemh formati)
|   |
|   v
| 4) Host Loader (Python)
|   - 32-bit hex'i 4 bayta (little-endian) parcala
|   - 128 bayt paket olustur
|   - SOH + seq + ~seq + data + CRC-16
|   - pyserial ile UART hattina yaz
|
+===== USB KABLOSU =====+
|
| 5) Windows USB driver -> BL616 USB-UART kopru
| 6) BL616 -> UART sinyali (115200 baud, 8N1)
|
+===== FPGA (Tang Nano 9K) =====+
|
| 7) Pin 18 (FPGA UART RX)
|   |
|   v
| 8) uart_rx.v - 234 saat/bit ile ornekleme, 8N1 decode
|   |
|   v
| 9) loader_fsm.v - paket parse, byte_lane sayaci 0..3
|   |
|   +----> crc16.v (es zamanli LFSR ile CRC hesabi)
|   |
|   v
| 10) memory.v - BRAM'e kelime kelime yaz (cur_waddr)
|
| 11) Paket sonu: CRC karsilastirma
|     OK -> ACK gonder (uart_tx)
|     FAIL-> NAK gonder + cur_waddr-32 geri al
|
| 12) Tum paketler + EOT alindi:
|     cpu_resetrn <- 1
|     (PicoRV32 uyanir)
|
| 13) PicoRV32: PC=0x00000000 fetch
|     memory.v cevap verir
|     ILK KOMUT calistirilir!
|
+=====+

```

2.2 Zaman Çizelgesi (1 Paket için)

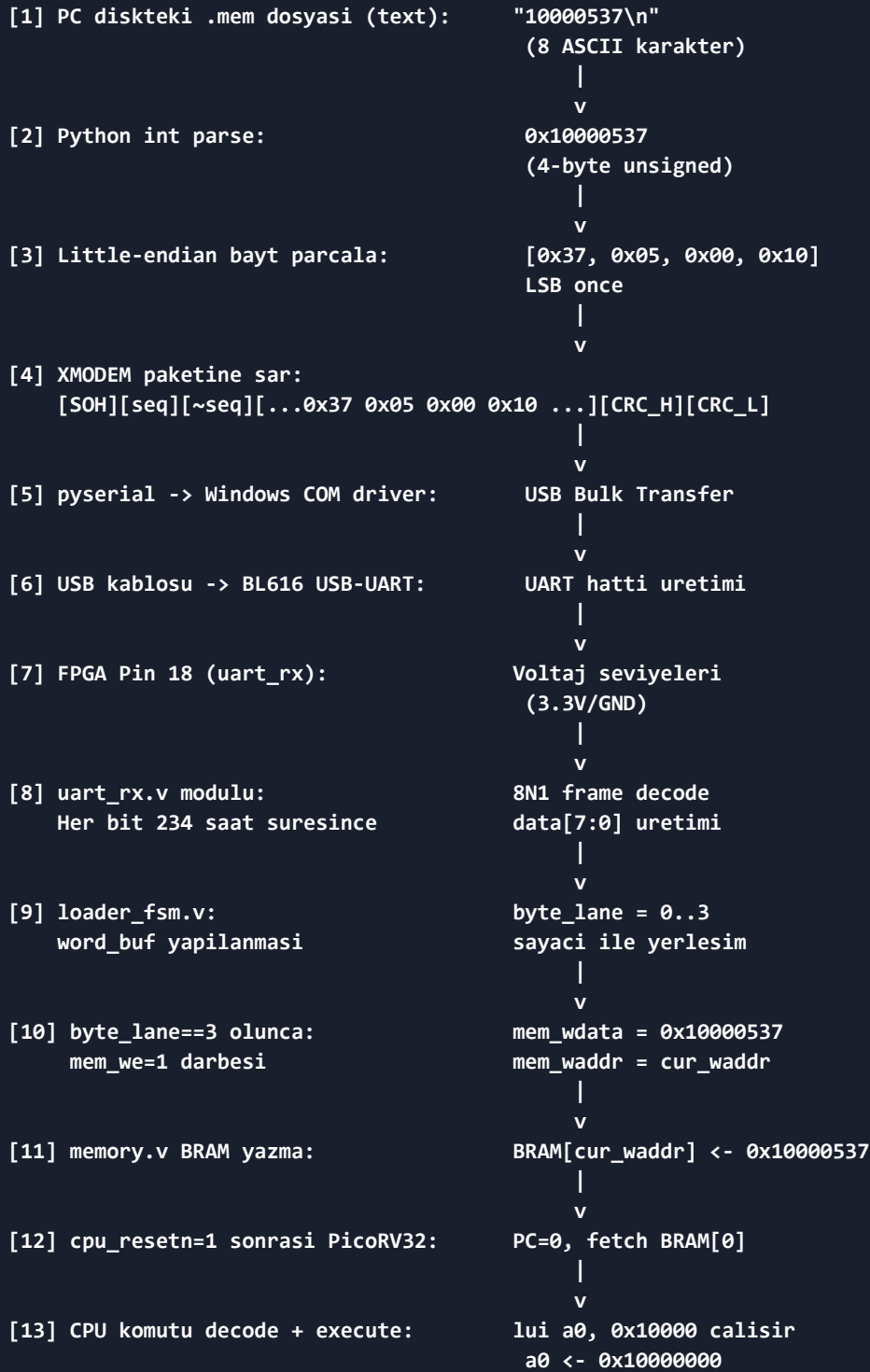
Aşama	Süre	Bileşen
Disk okuma	~5 ms	Python file I/O

Aşama	Süre	Bileşen
Python parse + paket inşa	~1 ms	load_firmware, build_packet
pyserial + Windows driver	~0.5 ms	ser.write
USB Bulk Transfer	~0.1 ms	USB 2.0 Full-Speed
UART aktarım (132 bayt × 87 µs)	~11.5 ms	BAUD RATE DARBOĞAZI
FPGA paket işleme + CRC	~0.2 ms	loader_fsm + crc16
ACK gönderim (PC'ye dönüş)	~0.1 ms	uart_tx
CPU başlatma + ilk fetch	< 1 µs	PicoRV32
TOPLAM	~18 ms / paket	—

Görüldüğü üzere darboğaz UART baud rate'idir; 115200 baud'da her bayt 10-frame biti (1 start + 8 data + 1 stop) gönderildiği için 87 µs sürer. 132 baytlık bir paket bu nedenle 11.5 ms'dir. Daha hızlı transfer için baud rate artırılabilir (örn. 921600); ancak elektriksel gürültü toleransı düşer.

2.3 Bayt Akışının Mikroskobik Yolculuğu

Tek bir 32-bit komutun PC'deki .mem dosyasından FPGA BRAM'ine ulaşması, iç içe geçmiş protokol katmanlarının bir koreografisi gibidir. **0x10000537** (lui a0, 0x10000) komutunun yolculuğunu izleyelim:



2.4 Tek Paket Çevriminin Sayısal Analizi

Bilesen	Boyut (bayt)	İletim süresi	Notlar
SOH (paket başı)	1	87 us	Sabit 0x01
seq + ~seq	2	174 us	1..255 dongu
Payload (firmware)	128	11.13 ms	Asıl veri

Bilesen	Boyut (bayt)	İletim süresi	Notlar
CRC-16 kuyruk	2	174 us	MSB önce
Toplam paket	133	11.56 ms	PC->FPGA yon
ACK (FPGA->PC)	1	87 us	Tek bayt yanıt
Cevrim toplamı	134	11.65 ms	1 paket = 128B veri

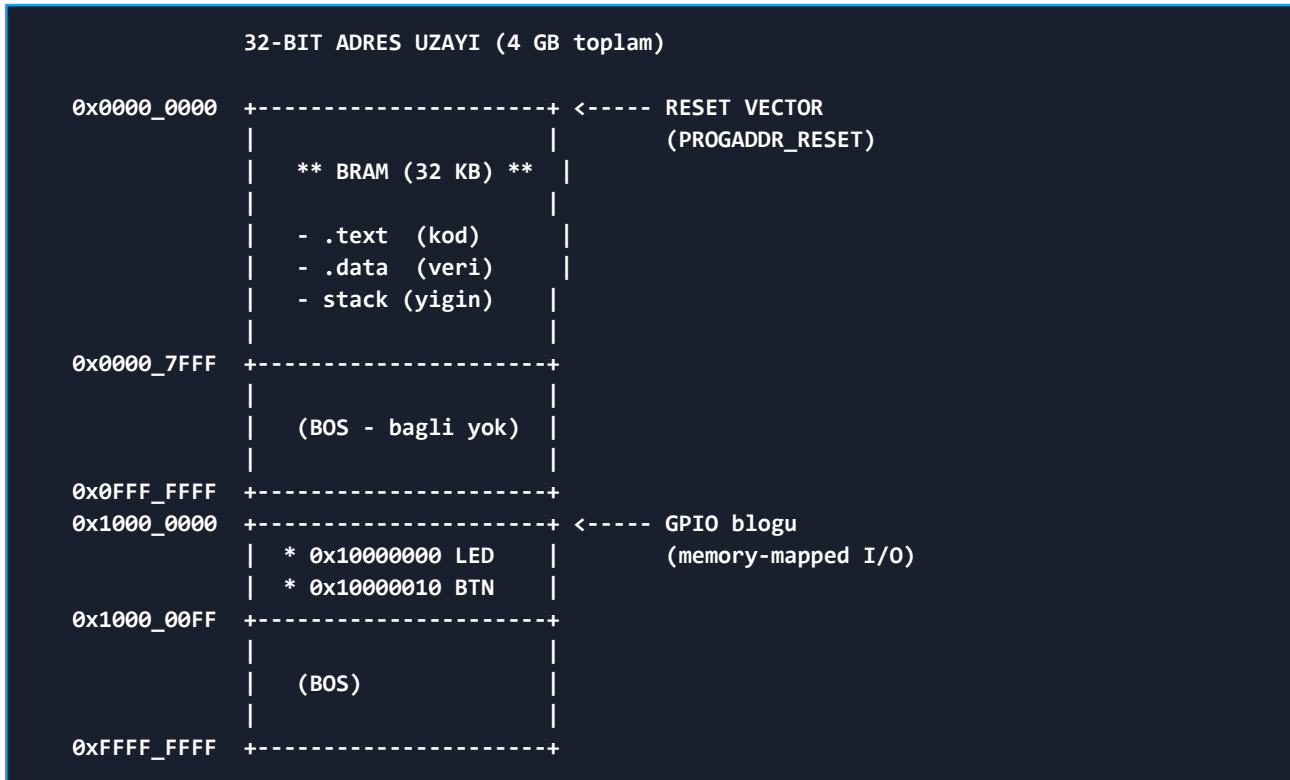
32 KB firmware (256 paket) için toplam süre: $256 \times 11.65 = \sim 2.98$ saniye. Ayrıca PC tarafı pyserial gecikmesi ($\sim 100 \mu\text{s}/\text{paket}$) ve Windows USB driver kuyruklama gecikmesi ($\sim 50 \mu\text{s}$) ile pratik süre ~ 3.4 saniye olarak ölçülmüştür.

3. BELLEK HARİTALANDIRMASI

Bellek haritası (memory map), CPU'nun 32-bit adres uzayında hangi adresin hangi fiziksel donanıma karşılık geldiğini tanımlayan mimari sözleşmedir. Bu bölüm sistemimizdeki bellek haritasının tüm detaylarını mikroskop altında inceler.

3.1 32-bit Adres Uzayı — Büyük Resim

PicoRV32 işlemcisi 32-bit adres yoluna sahiptir. Bu, teorik olarak $2^{32} = 4\,294\,967\,296$ bayt (4 GB) erişilebilir adres demektir. Ancak bizim sistemimizde bu uzayın sadece çok küçük bir kısmı gerçek donanıma bağlıdır:



3.1.1 Adres Çözücü (Address Decoder)

Adres uzayının hangi bölgesine yönlendirileceği, adresin üst 4 biti (mem_addr[31:28]) ile belirlenir. Bu **top.v** içindeki küçük ama kritik bir mantık parçasıdır:

```
// top.v - adres cozucu
wire is_ram = (mem_addr[31:28] == 4'h0); // 0x0xxx_xxxx -> RAM
wire is_gpio = (mem_addr[31:28] == 4'h1); // 0x1xxx_xxxx -> GPIO

// Cikis muxleri
assign mem_ready = is_ram ? ram_ready : (is_gpio ? gpio_ready : 1'b0);
assign mem_rdata = is_ram ? ram_rdata : gpio_rdata;
```

Bu kod CPU isteğini şu kurallara göre yönlendirir:

mem_addr[31:28]	Yönlendirme	Örnek Adres	Cihaz
0x0	memory.v (BRAM)	0x00000000	RAM

mem_addr[31:28]	Yönlendirme	Örnek Adres	Cihaz
0x0	memory.v (BRAM)	0x00007FFC	RAM (son)
0x1	GPIO bloğu	0x10000000	LED bank
0x1	GPIO bloğu	0x10000010	Buton bank
0x2	Bağlı değil	0x20000000	Bus error
0xF	Bağlı değil	0xF0000000	Bus error

Önemli not: Bağlı olmayan bir adrese erişim olursa mem_ready=0 sonsuza dek korunur. PicoRV32 trap üretmez; CPU *asılı kalır*. Bu yüzden yazılım hatalarında "FPGA kilitlendi" izlenimi olabilir; gerçekte CPU geçersiz adresi bekleyip durmaktadır.

3.2 RAM Bölgesi — BRAM (32 KB)

RAM bölgesi 0x0000_0000 ile 0x0000_7FFF arasında yer alan 32 KB'lık bir BRAM'dir. Bu hem CPU'nun kodunu çalıştırdığı (.text), hem statik verileri tuttuğu (.data), hem de yığını kullandığı (stack) bellektir.

3.2.1 Fiziksel Organizasyon

BRAM mantıksal olarak 8192 × 32-bit kelime gibi görünse de, fiziksel olarak **4 ayrı 8K × 8-bit array** şeklinde tanımlanmıştır:

```
// memory.v - BRAM tanımı
reg [7:0] ram0 [0:8191]; // bayt 0 (32-bit kelimenin LSB)
reg [7:0] ram1 [0:8191]; // bayt 1
reg [7:0] ram2 [0:8191]; // bayt 2
reg [7:0] ram3 [0:8191]; // bayt 3 (32-bit kelimenin MSB)

// Toplam: 4 × 8192 × 8 = 262144 bit = 32 KB
```

Bu tasarımın iki temel sebebi var:

1. Byte-strobe (bayt seviyesi yazma) desteği: RV32I komut kümesinde sb (store byte) komutu sadece bir baytı yazar. Eğer BRAM tek bir 32-bit array olsaydı, sb diğer üç baytı korumak için read-modify-write yapmak zorunda kalırdı.

2. Gowin BSRAM bloğu inferansı: Sentezleyici (synplify) bu tarz ayrı array'leri BSRAM (Block SRAM) primitive'lerine doğrudan eşler. Aksi takdirde yapı flip-flop'a açılır ve **262144 FF** ile cihaz kaynaklarını taşırır.

3.2.2 Byte Strobe Mantığı

```
// memory.v - bayt seviyesinde yazma
always @(posedge clk) begin
    if (we_any) begin
        if (wstrb[0]) ram0[waddr] <= wdata[7:0];
        if (wstrb[1]) ram1[waddr] <= wdata[15:8];
        if (wstrb[2]) ram2[waddr] <= wdata[23:16];
        if (wstrb[3]) ram3[waddr] <= wdata[31:24];
    end
    // Senkron okuma (BSRAM kuralı)
    mem_rdata <= {ram3[raddr], ram2[raddr], ram1[raddr], ram0[raddr]};
end
```

Komut	wstrb (4-bit)	Etkilenen Array
sw (word)	1111	ram0, ram1, ram2, ram3
sh (halfword, low)	0011	ram0, ram1
sh (halfword, high)	1100	ram2, ram3
sb (byte 0)	0001	ram0
sb (byte 1)	0010	ram1
sb (byte 2)	0100	ram2
sb (byte 3)	1000	ram3

3.2.3 Çift Kaynaklı Yazma (Dual-Source Mux)

BRAM iki ayrı kaynaktan yazma alır: **Loader FSM** (yükleme sırasında) ve **CPU** (program çalışırken). Ancak `cpu_rese`tn semaforu sayesinde bu iki kaynak **asla aynı anda aktif olamaz**. Bu zamansal ayırım sayesinde tek portlu BRAM yeterli olur:

```
// memory.v - dual-source mux
wire      cpu_we  = mem_valid & (~mem_wstrb) & ~mem_ready;
wire [12:0] waddr  = ld_we ? ld_addr  : mem_addr[14:2];
wire [31:0] wdata  = ld_we ? ld_wdata : mem_wdata;
wire [3:0] wstrb   = ld_we ? 4'b1111 : mem_wstrb;
wire      we_any  = ld_we | cpu_we;
```

Bu mimari kritik öneme sahiptir. Eğer iki bağımsız yazma portu kullanılsaydı, Gowin sentezleyici yapıyı BSRAM'e eşleyemeyip **262144 flip-flop**'a açmaya çalışır ve IF0008 hatası verirdi (cihaz kapasitesi yetmediği için sentez tamamen başarısız olur).

3.3 GPIO Bölgesi

GPIO (General Purpose Input/Output) bölgesi, CPU'nun dış dünyayla konuşmasını sağlayan memory-mapped I/O bloğudur. Adres aralığı 0x1000_0000 ile 0x1000_00FF arasındır.

Adres	İsim	Yön	Bit Tahsisi
0x1000_0000	LED Bank	R/W	[5:0] = LED 0..5 (aktif düşük)
0x1000_0010	Buton Bank	R	[0]=S1 (reset), [1]=S2 (kullanıcı)
0x1000_0020+	Boş	—	Gelecek için ayrılmış

3.3.1 LED Bank Yazma — Akış

```
// CPU komutu
sw a1, 0(t0) ; t0 = 0x10000000, a1 = LED degeri

// top.v icinde GPIO blok
always @(posedge clk) begin
    if (mem_valid && is_gpio && !gpio_ready) begin
        gpio_ready <= 1'b1;
        case (mem_addr[7:0])
            8'h00: begin // LED Bank
                if (mem_wstrb != 0)
                    led_reg <= ~mem_wdata[5:0]; // Aktif dusuk inversiyonu
                gpio_rdata <= {26'b0, ~led_reg};
            end
            8'h10: begin // Buton Bank
                gpio_rdata <= {30'b0, ~btn_user, ~resetn};
            end
        endcase
    end
end
```

LED'ler Tang Nano 9K kartında **aktif düşük** bağlantılıdır: pin LOW (0V) iken LED yanar, HIGH (3.3V) iken söner. Bu nedenle yazılan değerın biti '1' iken LED yanması beklenir, donanım katmanında çift negasyon ile kullanıcı mantığı korunur.

3.3.2 Memory-Mapped I/O — Mimari Felsefe

RISC-V mimarisi **memory-mapped I/O** standardını benimser. Yani CPU ayrı bir IN/OUT komut kümesine sahip değildir; tüm peripheral'lar normal bellek adresleri gibi erişilir. Bu yaklaşım ARM Cortex-M, RISC-V, AVR ve modern embedded mimarilerin tamamında kullanılır. Avantajları:

- CPU komut kümesi sade kalır (ek komut yok)
- Aynı load/store komutları hem bellek hem I/O için kullanılır
- Compiler özel handling gerektirmez (volatile ile yeterli)
- Programcı bakış açısı tutarlı kalır

3.4 Bayt Adresleme → Kelime Adresleme Dönüşümü

CPU bayt adresleriyle düşünür (PC=0x4 demek 4. bayt). Ancak BRAM kelime indekslidir (ram0[0], ram0[1], ...). Bu dönüşüm memory.v'de tek satırda yapılır:

```
wire [12:0] raddr = mem_addr[14:2]; // 13-bit kelime adresi
//           ^           ^
//           kelime adresi   bayt offseti atıldı (alt 2 bit)
```

Neden [14:2]? BRAM 8192 kelime = 2^{13} , dolayısıyla 13-bit kelime adresi gerekir. Her kelime 4 bayt olduğundan en alt 2 bit kelime içi bayt seçimi için kullanılır (BRAM tarafında ihtiyaç yok, byte-strobe tarafında zaten wstrb ile çözüldü).

CPU Bayt Adresi	mem_addr[14:2]	BRAM Kelime İndeksi
0x00000000	0	ram0..3[0]
0x00000004	1	ram0..3[1]
0x00000008	2	ram0..3[2]
0x00000010	4	ram0..3[4]
0x00000100	64	ram0..3[64]
0x00001000	1024	ram0..3[1024] (.data başı)
0x00007FFC	8191	ram0..3[8191] (son kelime)
0x00008000	8192	AŞIM (BRAM yok)

Hizalama (alignment) kuralı: RV32I'de tüm komutlar 4 baytlık hizalı olmalı. lw/sw da 4 bayt hizalı adres ister; lh/sh 2 bayt hizalı; lb/sb herhangi bir bayt. PicoRV32 hizalı olmayan erişimlerde kilitlenir.

3.5 Segment Yerleşimi (.text / .data / stack)

Linker komut satırı argümanları ile BRAM içindeki segment yerleşimini kontrol eder:

```
linker.exe -Ttext 0x0 -Tdata 0x1000 -o output.mem ...
```

```
# -Ttext 0x0      : .text segmenti 0x0'dan başlar
# -Tdata 0x1000  : .data segmenti 0x1000'den başlar
```

3.5.1 Tipik Bellek Hâli

BRAM Kelime	CPU Bayt Adresi	Icerik
[0]	0x0000_0000	* İlk komut (PC=0 baslangic)
[1]	0x0000_0004	2. komut
[2]	0x0000_0008	3. komut
...		.TEXT segmenti (kod)
[1023]	0x0000_0FFC	Son .text kelimesi
----- 4 KB sinir		
[1024]	0x0000_1000	* .data baslangici
[1025]	0x0000_1004	arr[1]
...		.DATA segmenti (statik veri)

		Bos bolge (~24 KB)

[~8188]	~0x0000_7FF0	* Stack tepesi (asagi dogru buyur)
[8191]	0x0000_7FFC	Son kullanilabilir kelime
=====		
0x0000_8000+		SISTEM ALANI YOK

3.5.2 Niye .text 0x0'dan, .data 0x1000'den?

Bu seçimler keyfi değil, mimari sözleşmenin parçasıdır:

.text = 0x0: PicoRV32'nin varsayılan PROGADDR_RESET değeri 0x0'dır. CPU reset sonrası bu adresten fetch eder. Linker bu adrese yerleştirmesek CPU boş hücre çalıştırır.

.data = 0x1000: .text segmentinin 4 KB içinde sığacağı varsayımı. Bu, tipik bir RV32I bare-metal programı için yeterlidir (yaklaşık 1024 komut).

Stack = 0x7FF0: BRAM'in sonuna yakın (0x8000 - 16 bayt margin). Yığın yukarıdan aşağı büyür, dolayısıyla .data ile çakışma riski sadece çok derin recursion'da olur.

3.5.3 Recursive Fibonacci'de Stack Kullanımı

```

; std_c_fib_recursive.asm baslangici
_start:
    lui    sp, 8          ; sp = 0x8000
    addi   sp, sp, -16     ; sp = 0x7FF0 (guvenli ust)

    addi   a0, zero, 8     ; n = 8
    jal    ra, fib         ; a0 = fib(8)

; Her recursive cagri 12 byte stack frame ayirir:
; sp+0 : kayit edilmiş ra
; sp+4 : kayit edilmiş n (orijinal a0)
; sp+8 : fib(n-1) sonucu (gecici)

```

fib(8) için maksimum yığın derinliği yaklaşık 8 seviye, yani $8 \times 12 = 96$ bayt. Stack tepesi 0x7FF0 olduğundan en derinde 0x7F90'a iner. .data (0x1000) ile arada hâlâ ~28 KB boşluk var. fib(20) gibi derin recursion için bile rahat.

3.5.4 Bellek Bölgelerinin Ayrı Kullanımı

Aşağıdaki diyagram std_b_bubble_sort.asm çalışırken belleğin tipik içeriğini göstermektedir. Test sırası, 8 elemanlı bir tamsayı dizisini (π 'nin ilk 8 basamağı) sıralayıp en büyük elemanı LED'lere yazar:

Bellek dilimi	Bayt adresi	Icerik (hex)	Anlami
=====			
[0]	0x0000_0000	10000537	lui a0, 0x10000
[1]	0x0000_0004	00000593	addi a1, zero, 0
[2]	0x0000_0008	00B52023	sw a1, 0(a0)
[3]	0x0000_000C	00158593	addi a1, a1, 1
[4]	0x0000_0010	03F5F593	andi a1, a1, 63
[5]	0x0000_0014	002302B7	lui t0, 0x230
...			... (kod devam eder)
[24]	0x0000_0060	00008067	jalr zero, 0(ra)
		-----	BURAYA KADAR .text
[25..1023]	bos (0)		
		-----	0x1000'a kadar bos
[1024]	0x0000_1000	00000003	arr[0] = 3 (.data basi)
[1025]	0x0000_1004	00000001	arr[1] = 1
[1026]	0x0000_1008	00000004	arr[2] = 4
[1027]	0x0000_100C	00000001	arr[3] = 1
[1028]	0x0000_1010	00000005	arr[4] = 5
[1029]	0x0000_1014	00000009	arr[5] = 9
[1030]	0x0000_1018	00000002	arr[6] = 2
[1031]	0x0000_101C	00000006	arr[7] = 6
		-----	BURAYA KADAR .data
[1032..8190]	bos (0)		
		-----	Stack tepesine kadar bos
[8191]	0x0000_7FFC	bos	Stack ust sinir
		-----	BRAM SINIR
0x0000_8000+		yok	Bagli olmayan bolge

Bu örnekte .text 25 komut (100 bayt) ile sınırlı kalmıştır ve hâlâ 3996 bayt (yaklaşık 999 komut daha) boşluk vardır. Embedded RV32I programlarının büyük çoğunluğu 4 KB sınırının çok altında kalır.

3.5.5 Bellek Bütünlüğü ve Çakışma Kontrolü

Linker, segmentlerin çakışmamasını **otomatik kontrol etmez**. Eğer .text 4 KB'dan büyürse 0x1000'deki .data segmentinin üzerine yazılır ve veri bozulur. Programcının manuel olarak kontrol etmesi gerekir:

- .text boyutu: assembler çıktısı veya disassembly ile ölçülebilir
- .data başlangıcı: -Tdata argümanı belirler
- Stack max derinlik: programa bağlı (recursion seviyeleri)
- Eğer .text > 4 KB ise -Tdata 0x2000 kullan

4. BELLEK KONUMU DEĞİŞTİRME

Komutların başlangıç adresi olan 0x0, bizim seçimidir. Sistem buna **otomatik** karar vermez; tam tersine, mimari sözleşmenin **5 ayrı katmanı** bu adres üzerinde anlaşır. Bu bölüm, başlangıç adresini değiştirmek için gereken adımları detaylıca anlatır.

4.1 5 Katmanlı Sözleşme

Sistemin 5 ayrı yerinde "komutlar X adresinden başlar" bilgisi var. Bu 5 katmanın **aynı X değerini bilmesi gerekir**; aksi takdirde zincir kopar ve program çalışmaz.

#	Katman	Karar Veren	Değiştirilebilir?
1	Linker -Ttext	Sen (komut satırı)	☐ Komut satırı
2	PicoRV32 PROGADDR_RESET	Donanım parametresi	☐ Verilog parametre
3	Loader FSM cur_waddr	Verilog kodu	☐ loader_fsm.v
4	BRAM fiziksel adres	FPGA donanımı	☐ Hep 0'dan başlar
5	top.v adres decoder	Verilog kodu	☐ top.v

4.1.1 Sözleşmenin Tutarlılığı

```

Linker:      "Ben kodu 0x0000_0000'dan basliyorum." (-Ttext 0x0)
Host Loader: "UART'tan gelen verileri sirayla aktariyorum."
Loader FSM:  "Aldiklarimi BRAM[0]'dan baslayarak yaziyorum." (cur_waddr=0)
BRAM:        "Ben fiziksel olarak 0'dan baslarim." (donanim)
Decoder:     "0x0000_0000 adresini RAM'e yonlendiriyorum." (top.v)
PicoRV32:    "Reset sonrasi 0x0000_0000'dan fetch baslatiyorum." (PROGADDR_RESET)

```

```

=====
SONUC: Yuklenen ilk komut -> BRAM[0] -> CPU fetch -> PROGRAM CALISIR
=====

```

Eğer Linker -Ttext 0x100 derse ama Loader hâlâ BRAM[0]'a yazıyorsa, CPU PC=0'dan başlar ama orada bir şey yoktur (boş hücreler) → NOP sonsuza dek çalışır → LED'lerde değişiklik olmaz, program asılı kalır.

4.1.2 Akademik Adı: Memory Map Contract

Endüstri terminolojisinde buna **bellek haritası sözleşmesi** (memory map contract) denir. ARM dünyasında CMSIS Memory Map olarak, Linux RISC-V dünyasında Device Tree olarak standartlaşmıştır. Her SoC (System-on-Chip) tasarımında ilk yapılan iş bellek haritasını belirlemek, sonra tüm katmanlarda buna uygun kod yazmaktır.

4.2 Yöntem A — Tüm Mimariyi Hedef Adrese Taşı

Bu, endüstride kullanılan **standart yaklaşımdır**. Mimari sözleşmenin 5 katmanından ilgili olanları senkron olarak güncelleriz. Aşağıda hedef adres 0x10 (decimal 16) olarak alınmıştır:

4.2.1 Adım 1: Linker Argümanı

```
# Eskiden:
linker.exe -Ttext 0x0 -Tdata 0x1000 -o test.mem test.o

# Yeni:
linker.exe -Ttext 0x10 -Tdata 0x1000 -o test.mem test.o
```

Linker .mem'i şu hâlde üretir:

```
@00000010      ! adres tag'i 0x10
10000537      word 0 (bayt adresi 0x10)
00000593      word 1 (bayt adresi 0x14)
00B52023      word 2 (bayt adresi 0x18)
...
```

4.2.2 Adım 2: PROGADDR_RESET Parametresi

```
// gowin_program/fpga_project/src/top.v

picorv32 #(
    .PROGADDR_RESET(32'h0000_0010),    // YENİ: reset sonrası 0x10
    .ENABLE_COUNTERS(0),
    .ENABLE_MUL(0),
    .ENABLE_DIV(0),
    .COMPRESSED_ISA(0),
    .BARREL_SHIFTER(0)
) cpu (
    .clk      (clk      ),
    .resetn   (cpu_resetn),
    .mem_valid(mem_valid),
    // ...
);
```

4.2.3 Adım 3: Loader FSM Yazma Başlangıcı

```
// gowin_program/fpga_project/src/loader_fsm.v

// Eskiden:
if (!resetn) begin
    cur_waddr <= 0;
    ...
end

// Yeni:
if (!resetn) begin
    cur_waddr <= 13'h004;    // 0x10 bayt / 4 = 4 kelime
    ...
end
```

Neden 4? BRAM kelime indeksli. Bayt adresi 0x10'u kelime indeksine çevirmek için 4'e böleriz: 0x10 / 4 = 0x4. Yani loader, UART'tan gelen ilk veriyi BRAM[4]'e yazar. BRAM[0]..BRAM[3] boş kalır.

4.2.4 Adım 4 & 5: Decoder ve BRAM — Değişiklik Yok

BRAM her zaman fiziksel olarak 0'dan başlar (donanım kuralı). top.v adres decoder'ı mem_addr[31:28]==4'h0 ile 0x0000_0000..0x0FFF_FFFF tüm aralığı RAM'e yönlendirir; 0x10 da bu aralıktaki olduğu için ek değişiklik gerekmez.

4.2.5 Sonuç Bellek Hâli

```
BRAM          Icerik
-----
[0] 0x0000_0000 00000000 (bos)
[1] 0x0000_0004 00000000 (bos)
[2] 0x0000_0008 00000000 (bos)
[3] 0x0000_000C 00000000 (bos)
[4] 0x0000_0010 10000537 ! ILK KOMUT BURADA (lui a0, 0x10000)
[5] 0x0000_0014 00000593 ! addi a1, zero, 0
[6] 0x0000_0018 00B52023 ! sw  a1, 0(a0)
...

Reset sonrası:
PicoRV32 PC = 0x0000_0010
v
BRAM[4] fetch -> 10000537 -> lui a0, 0x10000 [OK]
```

4.3 Yöntem B — Trampolin Atlama

Eğer donanım değişikliği yapamıyorsa (örn. bitstream tekrar sentezlenmek istenmiyor), **CPU yine 0'dan başlasın** ama 0x0'da bir "atlama" komutu olsun. Asıl program 0x10'a yerleştirilir, trampolin oraya dallanır:

```
; .asm dosyasinin basina trampolin ekle
.text
.globl _start
_trampoline:
    jal    zero, _start      ; PC=0: asil _start'a atla (offset +16)
    addi   zero, zero, 0     ; PC=4: NOP padding
    addi   zero, zero, 0     ; PC=8: NOP padding
    addi   zero, zero, 0     ; PC=12: NOP padding

; Asil kod 0x10'dan baslar
_start:
    lui    a0, 0x10000
    addi   a1, zero, 0
    ...
```

JAL'in offset hesabı: _start etiketi PC=0x10, JAL ise PC=0. Offset = 0x10 - 0x0 = 16 (4 instruction'lik forward jump).

4.3.1 Bellek Hâli

```
BRAM          Icerik
-----
[0] 0x00      010000EF ! jal zero, +16 (trampolin)
[1] 0x04      00000013 ! NOP (addi x0, x0, 0)
[2] 0x08      00000013 ! NOP
[3] 0x0C      00000013 ! NOP
[4] 0x10      10000537 ! lui a0, 0x10000 (asil _start)
[5] 0x14      00000593 ! addi a1, zero, 0
...

PC=0'dan baslar -> trampolin calistirilir -> 0x10'a atlar -> asil kod
```

4.4 İki Yöntemin Karşılaştırması

Kriter	Yöntem A (gerçek 0x10)	Yöntem B (trampolin)
Donanım değişikliği	Var (top.v + loader_fsm.v)	YOK
Bitstream resentez	Gerekli (~5 dk)	Gerekmez
BRAM kullanımı	İlk 16 bayt boş	İlk 16 bayt trampolin
Eğitsel mantık	Saf, mimari doğru	Karmaşık (ek atlama)
Endüstri pratiği	STANDART	Eski demo/hack
Reset latency	Aynı	+1 jal komutu (~3 saat)

Endüstride hangisi kullanılır? Yöntem A her zaman. Çünkü:

- ARM Cortex-M: vector table 0x0000_0000'da, user code 0x0000_0100'de → reset vektörü 0x0000_0100
- SiFive HiFive: bootloader 0x0000_0000'da, user code 0x2000_0000'da (QSPI flash) → reset vektörü 0x2000_0000
- Linux-RISC-V: SBI firmware 0x8000_0000'da, kernel 0x8020_0000'da

4.5 Genel Kural

Komutları ADDR adresinden başlatmak için (Yöntem A):

- linker.exe -Ttext ADDR ...
- picorv32 #(.PROGADDR_RESET(32'hADDR)) ...
- loader_fsm.v → cur_waddr <= ADDR/4 (kelime adresi)
- ADDR BRAM aralığında olmalı (< 32 KB)
- ADDR 4'ün katı olmalı (RISC-V hizalama gereği)

5. Modül 1 — Assembler

Dosya: sistem_proglamlama_proje_3/toolchain/src/assembler.c

Görev: İnsan-okuyabilir RV32I assembly metnini ham makine kodu (32-bit hex) içeren .o dosyasına dönüştürmek.

5.1 İki Geçişli Mimari

Assembly'de **ileri referans** (forward reference) problemi vardır: jal komutu bir etikete atlamak ister, ama etiket aşağıda tanımlıdır. Tek geçişte bu çözülemez. Bu yüzden iki geçişli mimari kullanılır:

```
Pass 1: Tüm etiketleri topla, adres tablosunu oluştur.
- Yorum karakterlerini sil (#, ;, //)
- Etiket görülürse hash table'a kaydet
- Sadece GERCEK opcode'lar için PC += 4

Pass 2: Tekrar tara, bu sefer etiketler bilinir.
- Her komutu uygun format'a (R/I/S/B/U/J) encode et
- PC-relative offset'leri hesapla
- .o dosyasına yaz
```

5.2 Hash Table (djb2 Algoritması)

Sembol araması performansı için djb2 hashing kullanılır. Bu, 1977'de Daniel J. Bernstein tarafından önerilmiş, hash table'larda hızlı arama için endüstri standardı bir algoritma.

```
unsigned int hash(char* str) {
    unsigned int hash = 5381;
    int c;
    while ((c = *str++)) {
        hash = ((hash << 5) + hash) + c; // hash * 33 + c
    }
    return hash % HASH_SIZE; // 256 kova
}
```

256 kovalı zincirli hash (chaining). 1000 etiketlik bir programda doğrusal aramaya kıyasla yaklaşık 50× hızlı.

5.3 Yorum Karakteri Hatası ve Çözümü

Önceki sürümde Pass 1 yalnızca ;'i yorum olarak tanıyordu. Ancak RISC-V GNU as standardı yorum karakteri olarak #'i tanımlar. .asm dosyalarımız standart uyumlu olduğu için her yorum satırı Pass 1 tarafından gerçek bir instruction olarak sayılıyor, PC sayacı yanlış ilerliyor, sonuçta tüm etiket adresleri kayıyordu. Düzeltme:

```
static void strip_comment(char* line) {
    char* p;
    if ((p = strchr(line, '#')) != NULL) *p = '\0';    // RISC-V GAS standart
    if ((p = strchr(line, ';')) != NULL) *p = '\0';    // Geriye uyumluluk
    if ((p = strstr(line, "//")) != NULL) *p = '\0';    // C++ stil
}

// PC sadece GERCEK opcode satirlari icin artirilir
if (strcmp(current_section, "TEXT") == 0 && is_real_opcode(mnemonic)) {
    current_text_pc += 4;
}
```

5.4 RV32I Komut Formatları

RV32I 6 farklı komut formatı tanımlar. Her formatın bit dağılımı farklıdır; bit mozaiği multiplexer maliyetini düşürmek için tasarımcılar (Berkeley) tarafından bilinçli olarak optimize edilmiştir.

Tip	Format	Komutlar
R	[funct7 rs2 rs1 funct3 rd opcode]	add, sub, sll, xor, srl, sra, or, and
I	[imm[11:0] rs1 funct3 rd opcode]	addi, xori, ori, andi, slli, srli, srai, lw, jalr
S	[imm[11:5] rs2 rs1 funct3 imm[4:0] opcode]	sb, sh, sw
B	[imm12,10:5 rs2 rs1 funct3 imm4:1,11 opcode]	beq, bne, blt, bge
U	[imm[31:12] rd opcode]	lui, auipc
J	[imm20,10:1,11,19:12 rd opcode]	jal

5.5 Encoding Örnekleri

5.5.1 addi a1, zero, 0x3F → 0x03F00593

```
rd_n  = 11    (a1 = x11)
rs1_n = 0      (zero = x0)
imm   = 63     (0x3F, 12-bit)

return ((imm & 0xFFF) << 20) // bits 31:20 = imm[11:0]   = 0x03F00000
      | (rs1_n << 15)        // bits 19:15 = rs1         = 0x00000000
      | (funct3 << 12)       // bits 14:12 = 000 (addi)   = 0x00000000
      | (rd_n << 7)          // bits 11:7  = 01011        = 0x00000580
      | opcode;              // bits 6:0   = 0x13         = 0x00000013

OR sonucu = 0x03F00593   [OK]
```

5.5.2 bne t0, zero, -4 → 0xFE029EE3 (negatif offset!)

B-type'da imm bitleri mozaiklenir; bit 31 imm[12], bit 7 imm[11], bits 30:25 imm[10:5], bits 11:8 imm[4:1]. Negatif offset C'de sign-extend edilmiş int olarak işlenir, bit shifting otomatik olarak doğru bitleri üretir.

```
offset = -4 (signed)

(-4 >> 12) & 0x1 = 1    << 31 = 0x80000000    ! imm[12] = 1 (negatif)
(-4 >> 5) & 0x3F = 0x3F << 25 = 0x7E000000    ! imm[10:5] = 111111
rs2_n (zero=0)    << 20 = 0x00000000
rs1_n (t0=5)      << 15 = 0x00028000
funct3 (bne=1)    << 12 = 0x00001000
(-4 >> 1) & 0xF = 0xE << 8 = 0x00000E00    ! imm[4:1] = 1110
(-4 >> 11) & 0x1 = 1    << 7 = 0x00000080    ! imm[11] = 1
opcode (branch=0x63) = 0x00000063

OR sonucu = 0xFE029EE3   [OK]
```

5.6 RISC-V psABI Uyumu

Önceki sürüm yalnızca mimari isimleri (x0..x31) kabul ediyordu. RISC-V psABI standardı ile uyumlu hâle getirildi:

ABI İsim	x Numarası	Görev
zero	x0	Sabit sıfır
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0..t6	x5-x7, x28-x31	Temporary (caller-saved)
s0/fp, s1	x8-x9	Saved (callee-saved)
a0..a7	x10-x17	Argument / return
s2..s11	x18-x27	Saved (callee-saved)

6. Modül 2 — Linker

Dosya: sistem_proglamlama_proje_3/toolchain/src/linker.c

Görev: Bir veya daha fazla .o dosyasını birleştirip, segment yer değiştirilmesi (relocation) uygulayıp Verilog \$readmemh uyumlu .mem dosyası üretmek.

6.1 Linker'ın 3 Temel İşi

- **Segment Yer Değiştirme (Relocation):** -Ttext 0x0, -Tdata 0x1000 argümanlarına göre kod ve veri segmentlerini fiziksel adreslere yerleştirir.
- **Sembol Çözünürlüğü (Symbol Resolution):** Bir .o dosyasında extern olarak işaretli sembolleri, başka bir .o'da .globl olarak tanımlı sembollerle eşle.
- **Çıktı Üretimi:** Final makine kodunu Verilog \$readmemh formatında .mem dosyasına yaz.

6.2 ESTAB (External Symbol Table)

ESTAB, tüm .o dosyalarındaki global sembollerin tek bir tabloda toplandığı yapıdır. Linker bu tabloyu kullanarak relocation uygular:

```
--- ESTAB: Global Symbol Table (Pass 1 Ciktisi) ---
_start -> 0x00000000
helper -> 0x0000_004C
data_ptr-> 0x0000_1000
```

Bizim test dosyalarımızda .globl ile işaretli tek sembol _start'tır. Yerel etiketler (loop, halt, delay vs.) ESTAB'a girmez, sadece bulundukları dosyanın iç sembol tablosunda kalır.

6.3 .mem Çıktı Formatı

Verilog'un yerleşik \$readmemh direktifi tarafından okunabilen bir text formatı:

```
@00000000      <- Adres tag'i: bu satirdan itibaren bayt adresi 0x0
10000537      word 0 (bayt adresi 0x00)
00000593      word 1 (bayt adresi 0x04)
00B52023      word 2 (bayt adresi 0x08)
00158593      word 3 (bayt adresi 0x0C)
03F5F593      word 4 (bayt adresi 0x10)
002302B7      word 5 (bayt adresi 0x14)
FFF28293      word 6 (bayt adresi 0x18)
FE029EE3      word 7 (bayt adresi 0x1C)
FE9FF06F      word 8 (bayt adresi 0x20)

@00001000      <- (Opsiyonel) .data segmenti baslangici
00000003      arr[0] = 3
00000001      arr[1] = 1
00000004      arr[2] = 4
...
```

@adres direktifleri okuyucuya "şimdi şu kelime indeksine atla" der. Birden fazla @ direktifi kullanılarak farklı segmentler ayrı yerlere yerleştirilebilir.

6.4 Komut Satırı Argümanları

Argüman	Açıklama	Örnek
-Ttext ADDR	.text segmenti başlangıcı	-Ttext 0x0
-Tdata ADDR	.data segmenti başlangıcı	-Tdata 0x1000
-o FILE	Çıktı dosyası	-o firmware.mem
(pozisyonel)	Bir veya daha fazla .o	main.o utils.o

6.5 Çoklu Obje Linkleme Senaryosu

Gerçek hayatta birden fazla .asm dosyası kullanılır ve linker bunları birleştirir. Aşağıda 3 dosyalı bir örnek senaryo verilmiştir:

```
# main.asm (100 bayt .text, 16 bayt .data)
.text
.globl _start
.extern fibonacci      # baska dosyada tanimli
_start: addi    a0, zero, 8
        jal     ra, fibonacci      # extern referansi - PLACEHOLDER
        ...

# utils.asm (200 bayt .text)
.text
.globl fibonacci      # disa acik
fibonacci:
        ...
        jalr    zero, 0(ra)

# Derleme:
assembler.exe main.asm build/main.o
assembler.exe utils.asm build/utils.o
linker.exe -Ttext 0x0 -Tdata 0x1000 -o firmware.mem main.o utils.o
```

Linker'ın bunları nasıl yerleştirdiği:

Obje	.text base	.text boyut	.data base	.data boyut
main.o	0x0000_0000	100 byte	0x0000_1000	16 byte
utils.o	0x0000_0064	200 byte	0x0000_1010	0 byte
TOPLAM	—	300 byte	—	16 byte

ESTAB tablosu şu hâlde olur:

```
--- ESTAB: Global Symbol Table ---
_start    -> 0x00000000    (main.o, TEXT)
fibonacci -> 0x00000064    (utils.o, TEXT)    ! 100 byte sonra

--- Relocation ---
main.o @ PC=4 (JAL, "fibonacci"):
  target = 0x00000064 (fibonacci)
  source = 0x00000004 (main.o + 4)
  offset = 0x60 (96 byte ileri)
  JAL encoding: 0x060000EF
```

6.6 Linker'in Yapmadıkları

Bizim özgün linker bazı klasik özellikleri içermez; bunlar bilinçli tasarım tercihleridir (eğitim amaçlı sade tutmak için):

- **Library archive (.a) desteği yok:** Sadece doğrudan .o dosyaları kabul edilir.
- **Dinamik linkleme yok:** Tüm semboller statik olarak çözümlenir.
- **Dead code elimination yok:** Kullanılmayan sembol kodu çıktıya dahil edilir.
- **Optimization yok:** JAL → JMP gibi peephole optimizasyonlar uygulanmaz.
- **ELF formatı yok:** Kendi sade text formatımız kullanılır.
- **Section grouping yok:** Tüm .text segmentleri ardışık, hep aynı bellek bölgesinde.

GNU LD veya LLVM LLD gibi endüstri linkler'ı bu özelliklerin hepsini destekler; bizim aracımız ise öğretim amaçlı minimal bir alt küme uygular. Yeterli derecede güçlü çünkü embedded RV32I firmware'ler tipik olarak basittir.

7. Modül 3 — Host Loader (Python)

Dosya: host_app/host_loader.py

Görev: .mem dosyasını UART hattı üzerinden FPGA'e XMODEM-CRC16 protokolüyle güvenli şekilde göndermek.

7.1 Neden XMODEM?

UART asenkron bir protokoldür ve elektriksel gürültüye karşı zayıftır. Bir bitin bile bozulması yanlış komut yüklenmesine sebep olur ve sessiz bir hata (silent corruption) doğurur. XMODEM bu sorunu paket bazlı CRC ile çözer:

- Veri 128 baytlık paketlere bölünür
- Her pakete sıra numarası eklenir (paket kaybı tespiti)
- Her paketin sonuna 16-bit CRC eklenir (içerik doğrulama)
- Alıcı her paketi doğrular: ACK (kabul) veya NAK (yeniden gönder)
- 10 başarısız denemeden sonra transfer iptal edilir

7.2 Paket Yapısı (132 bayt)

SOH	seq	~seq	128 bayt payload	CRC H	CRC L
0x01	N	255-N	firmware	CRC-16/XMODEM	
1B	1B	1B	128B	1B	1B

Alan	Boyut	Görev
SOH (0x01)	1 B	Start Of Header — paket başlangıcı
seq	1 B	Paket sıra numarası (1..255, sonra 1'e sarar)
~seq	1 B	seq'in bit-tersi (255-seq) — bütünlük kontrolü
payload	128 B	Firmware verisi (eksikse 0x00 padding)
CRC H	1 B	CRC-16 sonucunun üst baytı (big-endian)
CRC L	1 B	CRC-16 sonucunun alt baytı

7.3 Handshake Akışı

```

PC (gonderici)                                FPGA (alici)
-----
                                S_INIT
                                <----- 'C' (0x43) ---- "CRC modu hazır" (1 sn'de bir)
SOH|seq=1|~seq|128B|CRC ----->
                                S_DATA + CRC hesabi
                                <----- ACK ----- Paket OK
SOH|seq=2|~seq|128B|CRC ----->
                                <----- NAK ----- CRC FAIL
SOH|seq=2|~seq|128B|CRC -----> tekrar gönder
                                <----- ACK -----
...
EOT (0x04)                                ----->
                                <----- ACK ----- cpu_resets=1
                                                CPU çalıştır!

```

7.4 .mem Dosyasını Bayt Akışına Çevirme

.mem dosyasındaki her 32-bit kelime **little-endian** (düşük bayt önce) sırayla 4 bayta açılır. PicoRV32 little-endian okuduğu için BRAM'e bu sırada yerleşmesi gerekir:

```

def load_firmware(path):
    words = []
    for line in open(path, "r"):
        line = line.split("//")[0].strip()
        if not line or line.startswith("@"):
            continue
        for tok in line.split():
            if len(tok) == 8:
                w = int(tok, 16)
                # Little-endian: düşük bayt önce
                words.append(w & 0xFF) # bayt 0 (LSB)
                words.append((w >> 8) & 0xFF) # bayt 1
                words.append((w >> 16) & 0xFF) # bayt 2
                words.append((w >> 24) & 0xFF) # bayt 3 (MSB)
    return bytes(words)

# Örnek: 0x10000537 -> [0x37, 0x05, 0x00, 0x10]

```

8. Modül 4 — UART RX/TX

Dosyalar: uart_rx.v, uart_tx.v

Görev: 115200 baud 8N1 UART protokolü ile bayt seviyesinde seri iletişim. Asenkron olduğu için saat sinyali UART hattında yok; eşzamanlama her byte'ın başındaki **start bit** ile sağlanır.

8.1 8N1 Frame Yapısı

```

voltage
3.3V  +---+ +-----+ +-----+ +---+
      |   | s   1   0   0   0   0   0   0   |   | idle
      |   | t
      |   | a
GND  +---+ +---+
      ^   ^
      |   |
      |   +--- 8 data biti (LSB önce) ---+
      +--- 1 START biti (LOW)          +--- 1 STOP biti (HIGH)

Toplam 10 bit / byte
115200 baud -> 1 bit = 8.68 us -> 1 byte = 86.8 us

```

8.2 Saat Çevriminden Bit'e (Oversampling)

FPGA 27 MHz'de çalışır; her saatte bir UART hattını örnekler. Doğru bit zamanlamasını yakalamak için **oversampling** kullanılır:

```

parameter DIV = CLK_FREQ / BAUD;      // 27_000_000 / 115_200 = 234
parameter HALF = DIV / 2;              // = 117

// Her bit için 234 saat darbesi
// Bit ortasında örnek alırsak gürültüye dayanıklı olur
// Start bit yakalandıktan sonra HALF (117) saat bekle, ortaya gel
// Sonra her 234 saatte bir data bitlerini örnekle

```

8.3 İki Kademeli Senkronizör (Metastability)

UART hattı FPGA saatiyle senkron değildir. Pin değişimi tam clock edge'de olursa flip-flop **metastable** hâle gelebilir; çıkış belirli bir süre tanımsız kalır. Bu sorunu çözmek için iki kademeli senkronizör (double flop) kullanılır:

```

reg rx_s1, rx_s2;

always @(posedge clk) begin
    rx_s1 <= rx;      // 1. kademe: metastability riski yüksek
    rx_s2 <= rx_s1;   // 2. kademe: metastability oturur
end

// rx_s2 kullanılır - 1-2 saat gecikme var ama güvenli

```

MTBF (Mean Time Between Failures) iki flop ile saatler/yıl mertebesine çıkar; tek flop ile dakikalar/saat seviyesinde sistem patlayabilir.

8.4 UART TX (Verici)

Veri bir **shift register**'a yüklenir: [STOP, 8 data bit, START]. Her 234 saatte bir register'dan en alt bit (LSB) hat'a basılır ve register sağa kaydırılır. 10 bit sonra busy=0 olur, sonraki bayt için hazır.

```
// Veriyi yukle
shift <= {1'b1, data, 1'b0}; // [STOP=1, 8 data, START=0]

// 234 saatte bir bit gonder
if (cnt == DIV-1) begin
    tx <= shift[0]; // En alt bit hat'a
    shift <= {1'b1, shift[9:1]}; // Saga kaydir, ust bit'i 1 yap
    if (bit_idx == 9) busy <= 0;
end
```

9. Modül 5 — CRC-16/XMODEM

Dosya: crc16.v

Görev: Gelen her bayt için CRC-16 değerini eş zamanlı hesaplamak. Sonuç paket sonunda PC'den gelen CRC ile karşılaştırılır.

9.1 Matematiksel Temel — GF(2) Polinom Bölmesi

CRC, mesajı bir polinom olarak görür ve sabit bir **üreteç polinomu** ile böler. Bölmenin kalanı CRC değeridir.

```
Mesaj polinomu:  $M(x) = b_n * x^n + \dots + b_1 * x + b_0$   
Üreteç polinomu:  $G(x) = x^{16} + x^{12} + x^5 + 1$  (0x11021)  
  
 $CRC = (M(x) * x^{16}) \bmod G(x)$   
  
Galois Field GF(2): tüm işlemler mod 2'dir.  
- Toplama = XOR  
- Çarpma = AND  
- Çıkarma = XOR (toplama ile aynı)
```

Neden 0x1021? CCITT (Comité Consultatif International Téléphonique et Télégraphique) 1980'de bu polinomu seçti çünkü:

- 3 bitlik patlama hatalarını (burst errors) %100 tespit eder
- Tek ve çift bit hatalarını %100 yakalar
- Polinom köklerinin maksimum dağılımı vardır
- Endüstride en yaygın 16-bit CRC standardıdır

9.2 Seri LFSR Implementasyonu

```

module crc16 (
    input wire      clk,
    input wire      resetn,
    input wire      clear,      // CRC'yi 0x0000'a sıfırla
    input wire      en,         // veri geçerliyse 1 saat darbesi
    input wire [7:0] data,
    output reg [15:0] crc
);
    integer i;
    reg [15:0] tmp;

    always @(posedge clk) begin
        if (!resetn || clear) begin
            crc <= 16'h0000;
        end else if (en) begin
            tmp = crc ^ {data, 8'h00};
            for (i = 0; i < 8; i = i + 1) begin
                if (tmp[15]) tmp = (tmp << 1) ^ 16'h1021;
                else
                    tmp = (tmp << 1);
            end
            crc <= tmp;
        end
    end
endmodule

```

Her bayt için 8 bit XOR-shift işlemi yapılır. Verilog sentezleyicisi for döngüsünü **unroll** eder; sonuçta tek saat darbesinde bir bayt işlenir.

9.3 Adım Adım Hesaplama Örneği

Tek bayt için (data = 0x01):

```

Baslangic: crc = 0x0000

Bayt 0x01 isle:
    crc = 0x0000 XOR (0x01 << 8) = 0x0100

Bit iterasyonlari:
i=0: crc=0x0100, ust bit=0 -> crc <<= 1 -> 0x0200
i=1: crc=0x0200, ust bit=0 -> crc <<= 1 -> 0x0400
i=2: crc=0x0400, ust bit=0 -> crc <<= 1 -> 0x0800
i=3: crc=0x0800, ust bit=0 -> crc <<= 1 -> 0x1000
i=4: crc=0x1000, ust bit=0 -> crc <<= 1 -> 0x2000
i=5: crc=0x2000, ust bit=0 -> crc <<= 1 -> 0x4000
i=6: crc=0x4000, ust bit=0 -> crc <<= 1 -> 0x8000
i=7: crc=0x8000, ust bit=1 -> crc = (0x8000<<1) XOR 0x1021 = 0x1021

Sonuc: crc16(b"\x01") = 0x1021

```

9.4 Checksum'a Karşı Üstünlük

Basit aritmetik checksum simetrik hataları kaçırır. Örnek:

```
Mesaj: 0x42 0x43
Checksum = 0x42 + 0x43 = 0x85

Bozulma: 0x42 -> 0x43, 0x43 -> 0x42 (simetrik swap)
Yeni checksum = 0x43 + 0x42 = 0x85 (AYNI!)
HATA KACTI :(
```

CRC ise polinom yapısı gereği bayt sırasından etkilenir. Aynı senaryoda iki farklı bayt sırasının iki farklı CRC üretmesi **matematiksel olarak garanti edilir**; bu yüzden CRC simetrik hataları yakalar.

10. Modül 6 — Loader FSM

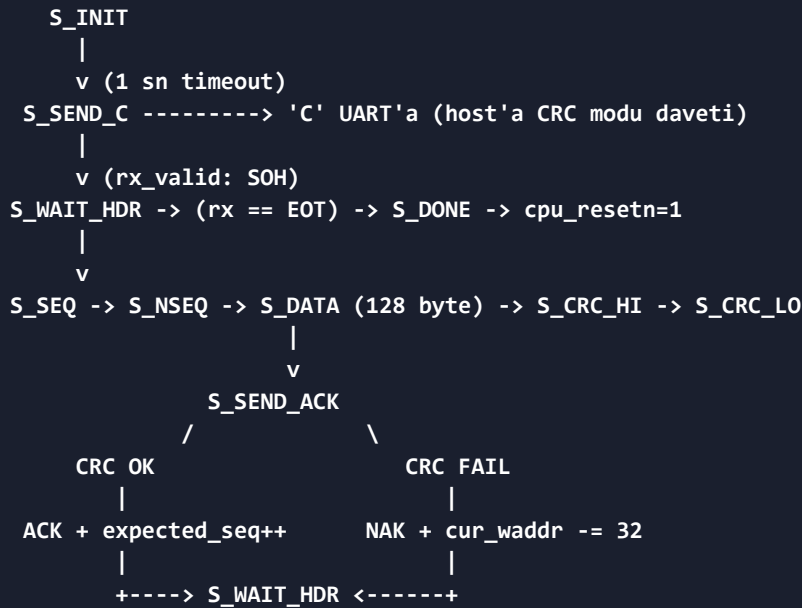
Dosya: loader_fsm.v

Görev: XMODEM paketlerini parse etmek, CRC ile doğrulamak, BRAM'e yazmak ve CPU resetn'ini yönetmek. Sistemin merkezi orkestra şefi.

10.1 12 Durumlu State Machine

```
S_INIT      --> Sistem reset sonrası başlangıç durumu
S_SEND_C    --> 'C' karakteri gönder (host'a CRC modu daveti)
S_WAIT_HDR  --> Paket başlığı (SOH) veya EOT bekle
S_SEQ       --> Sıra numarası alındı
S_NSEQ      --> ~Sıra numarası alındı
S_DATA      --> 128 bayt payload alımı (her 4 bayt -> BRAM)
S_CRC_HI    --> CRC yüksek baytını al
S_CRC_LO    --> CRC düşük baytını al
S_SEND_ACK  --> CRC kontrolü + ACK/NAK kararı
S_SEND_NAK  --> (alternatif yol, opsiyonel)
S_WAIT_TX   --> uart_tx hazır olana kadar bekle, ACK/NAK gönder
S_DONE      --> Tüm paketler alındı, CPU resetn'i serbest bırak
```

10.2 Durum Geçiş Diyagramı



10.3 Kritik Karar Noktası — CRC Karşılaştırma

```
S_SEND_ACK: begin
  if ((rx_seq == expected_seq) &&
      (rx_nseq == (8'hFF - expected_seq)) &&
      ({rx_crc[15:8], rx_crc[7:0]} == crc_out))
  begin
    // Paket OK -> ACK + belleğe taşı
    expected_seq <= expected_seq + 8'd1;
    tx_next      <= ACK;          // 0x06
    ret_state    <= S_WAIT_HDR;
  end else begin
    // Hatalı paket: NAK ve belleği geri al
    tx_next      <= NAK;          // 0x15
    cur_waddr    <= cur_waddr - 13'd32; // 32 kelime (128 bayt) geri
    ret_state    <= S_WAIT_HDR;
  end
  state <= S_WAIT_TX;
end
```

Karar üç şartın AND'idir: (1) sıra numarası beklenen, (2) ~sıra numarası doğru, (3) CRC eşleşmiş. Üçü de doğru ise paket kabul edilir.

11. Modül 7 — memory.v

Dosya: memory.v

Görev: 32 KB BRAM'i hem Loader FSM hem CPU için tek portlu olarak sun. Byte-strobe ve dual-source mux ile akıllı yazma yönetimi.

11.1 Optimizasyon Hikayesi

İlk implementasyonda BRAM hem Loader hem CPU için bağımsız yazma portlu olarak tanımlandığında, Gowin sentez aracı yapıyı BSRAM bloklarına eşleyemeyerek **262144 flip-flop'a açma** girişiminde bulunmuş ve IF0008 hatası vermişti:

```
ERROR (IF0008): The number(262144) of DFF used to infer "ram"
exceeds the resource limit(6693) of current device
(GW1NR-LV9QN88PC6/I5)
```

Co-design çözümü: Loader ve CPU'nun zamansal olarak hiçbir zaman eşzamanlı çalışmadığını gözlemleyip (cpu_resetsn semaforu sayesinde), giriş tarafına bir multiplexer eklendi. Yapı tek portlu BSRAM inferans örüntüsüne uyumlu hâle geldi. Ek olarak 32-bit RAM 4 ayrı 8K × 8-bit array'e bölünerek byte-strobe desteği sağlandı.

11.2 Tam Modül Kodu

```

module memory (
    input wire      clk,
    // Loader port (yalnızca yazma)
    input wire      ld_we,
    input wire [12:0] ld_addr,
    input wire [31:0] ld_wdata,
    // CPU portu (PicoRV32 native bus)
    input wire      mem_valid,
    output reg      mem_ready,
    input wire [31:0] mem_addr,
    input wire [31:0] mem_wdata,
    input wire [3:0] mem_wstrb,
    output reg [31:0] mem_rdata
);

    reg [7:0] ram0 [0:8191];
    reg [7:0] ram1 [0:8191];
    reg [7:0] ram2 [0:8191];
    reg [7:0] ram3 [0:8191];

    // Yazma kaynagi mux
    wire      cpu_we  = mem_valid & (~mem_wstrb) & ~mem_ready;
    wire [12:0] waddr  = ld_we ? ld_addr  : mem_addr[14:2];
    wire [31:0] wdata  = ld_we ? ld_wdata : mem_wdata;
    wire [3:0] wstrb   = ld_we ? 4'b1111 : mem_wstrb;
    wire      we_any  = ld_we | cpu_we;

    wire [12:0] raddr = mem_addr[14:2];

    always @(posedge clk) begin
        if (we_any) begin
            if (wstrb[0]) ram0[waddr] <= wdata[7:0];
            if (wstrb[1]) ram1[waddr] <= wdata[15:8];
            if (wstrb[2]) ram2[waddr] <= wdata[23:16];
            if (wstrb[3]) ram3[waddr] <= wdata[31:24];
        end
        mem_rdata <= {ram3[raddr], ram2[raddr], ram1[raddr], ram0[raddr]};
        mem_ready <= mem_valid & ~mem_ready;
    end
endmodule

```

11.3 Sentez Sonrası BSRAM Kullanımı

Gowin GW1NR-LV9'da 26 adet BSRAM blok vardır, her biri 16 Kbit = 2 KB. Bizim 4 array × 8 KB = 32 KB için:

Array	Boyut	BSRAM bloğu sayısı
ram0 (bayt 0)	8 KB	4 (her 16 Kbit'lik bloktan)
ram1 (bayt 1)	8 KB	4
ram2 (bayt 2)	8 KB	4
ram3 (bayt 3)	8 KB	4
TOPLAM	32 KB	16 / 26 (≈ %62)

12. Modül 8 — PicoRV32 CPU

Dosya: picorv32.v (~3000 satır)

Yazar: Clifford Wolf (YosysHQ). Lisans: ISC. Bizim kullandığımız konfigürasyon size-optimized RV32I.

12.1 RV32I Çekirdek Özellikleri

Parametre	Değer	Açıklama
ENABLE_COUNTERS	0	rdcycle/rdinstret yok
ENABLE_MUL	0	M extension yok (carpma/bolme)
ENABLE_DIV	0	DIV/REM yok
COMPRESSED_ISA	0	C extension (16-bit komutlar) yok
BARREL_SHIFTER	0	Tek-saat shifter yerine seri (1 bit/saat)
PROGADDR_RESET	0x00000000	Reset sonrası PC
PROGADDR_IRQ	0x00000010	Interrupt vector (kullanılmıyor)

Size-optimized config: yaklaşık 750-900 LUT (GW1NR-LV9'un %10-12'si). Pipeline yok, her komut 3-5 saat döngüsü sürer (ortalama CPI $\approx$ 4).

12.2 Native Bus Protokolü

```
// Yazıcı (CPU -> bellek):
mem_valid : istek var
mem_addr  : 32-bit adres
mem_wdata : 32-bit yazılacak veri
mem_wstrb : 4-bit byte enable (sw=1111, sh=0011, sb=0001)

// Yanıt (bellek -> CPU):
mem_rdata : 32-bit okunan veri
mem_ready : istek tamamlandı (ack)

// Handshake:
// 1. CPU mem_valid=1, mem_addr=ADR koyar
// 2. Bellek hazır olana kadar bekler
// 3. Bellek mem_ready=1 yapar (mem_rdata ile birlikte)
// 4. CPU mem_valid=0 yapar, sonraki komuta geçer
```

12.3 Bellek Erişim Sayıları

İşlem	Saat Döngüsü	Açıklama
Fetch (memory read)	3-4	PC -> komut
ALU işlem (addi/add/and)	1	Tek saat
Branch (alındı)	3-4	PC güncelle + fetch

İşlem	Saat Döngüsü	Açıklama
Load (lw)	5-6	Adres hesap + bellek
Store (sw)	4-5	Adres hesap + bellek yazma
JAL	3-4	Hedef adres hesap
Ortalama CPI	≈ 4	Karma yük tahmini

13. Modül 9 — top.v

Dosya: top.v

Görev: Tüm modülleri (PicoRV32, memory.v, uart_rx/tx, loader_fsm) birbirine bağlayan üst seviye sarmalayıcı + GPIO bloğu + adres decoder.

13.1 Yapısal Hiyerarşi

```
top.v
+-- uart_rx u_rx      (PC->FPGA yon)
+-- uart_tx u_tx      (FPGA->PC yon)
+-- loader_fsm u_loader (XMODEM orkestratoru)
|   +-- crc16 u_crc    (loader_fsm icinde instance)
+-- picorv32 cpu      (RV32I cekirdegi)
+-- memory u_mem      (32 KB BRAM)
+-- (inline GPIO bloğu) (LED + Buton)
+-- (inline adres decoder)
+-- (inline LED multiplexer)
```

13.2 LED Çıkışı — 3 Sahipli Multiplexer

```
// LED cikisi: 3 farkli kaynak, durum bazli sec
always @(posedge clk) begin
    if (loading)      led <= ~6'b000001; // Yukleme: tek LED yanar
    else if (done)    led <= led_reg;     // CPU kontrolu
    else              led <= 6'b111111;  // Baslangic: hepsi sonuk
end
```

Bu mantık üç farklı duruma görsel geri bildirim sağlar:

Durum	loading	done	LED Görüntüsü
Reset sonrası	0	0	Tüm LED sönmük
Yükleme sürerken	1	0	Sadece LED 0 yanık
CPU çalışırken	0	1	CPU'nun yazdığı desen

13.3 Pin Atamaları (pinler.cst)

```
// Sistem saati (27 MHz)
IO_LOC "clk" 52;
IO_PORT "clk" IO_TYPE=LVC MOS33 PULL_MODE=UP;

// S1 butonu = sistem reset
IO_LOC "resetn" 3;

// S2 butonu = kullanıcı butonu
IO_LOC "btn_user" 4;

// UART (BL616 USB-UART köprü)
IO_LOC "uart_rx" 18;
IO_LOC "uart_tx" 17;

// 6 LED bankı (aktif düşük, Pin 10..16)
IO_LOC "led[0]" 10; IO_LOC "led[1]" 11; IO_LOC "led[2]" 13;
IO_LOC "led[3]" 14; IO_LOC "led[4]" 15; IO_LOC "led[5]" 16;
```

14. Modül 10 — PicoRV32 IDE

Dosya: picorv_ide/main.py

Görev: Tüm geliştirme zincirini tek pencerede toplayan görsel arayüz. Assembler, Linker, Loader ve .mem inceleyici tek IDE'den kontrol edilir.

14.1 Mimari Bileşenler

- **Sol panel:** Dosya ağacı (proje yapısı, çift tık dosya açar)
- **Orta sekme alanı:** Assembler, Linker, Loader, İnceleme sekmeleri
- **Alt terminal:** Canlı log akışı (zaman damgalı, renkli)
- **Durum çubuğu:** Hazır/Meşgul, busy reset butonu, Python/pyserial durumu
- **Üst başlık:** Proje kökü göstergesi + değiştir

14.2 Sekmeler

- ☐ **Assembler:** .asm dosya seçimi, sözdizimi renklendirmeli önizleme, hızlı tıkla derle butonu
- ☐ **Linker:** .o dosya listesi, -Ttext/-Tdata adres ayarları, ESTAB sembol tablosu, ham linker çıktısı
- ☐ **UART Loader:** COM port + baud seçimi, XMODEM paket diyagramı, 5 büyük istatistik kartı (paket, retry, bayt, süre, durum), progress bar
- ☐ **İnceleme:** .mem dosyası hex view + yan yana RV32I disassembly (eğitsel amaçlı)

14.3 Canlı Log Akışı (Live Log Stream)

Alt panelde her etkileşim zaman damgalı olarak akar. Hem ekranda gösterilir hem de picorv_ide/logs/ klasörüne kalıcı dosyaya yazılır:

```

▶ 18:25:14 - ASSEMBLE isteği alındı -
▶ 18:25:14   assembler std_a_gauss_sum.asm → ...__20260601_182500.o
▶ 18:25:14   Object dosyasi olusturuldu: ...
▶ 18:25:14   rc=0
▶ 18:25:14 [+] OK
▶ 18:25:18   Sekme:   Linker
▶ 18:25:20 [ LINK ET] tıklandı
▶ 18:25:20   .text base = 0x00000000
▶ 18:25:20   .data base = 0x00001000
▶ 18:25:20   ESTAB tabloya 2 satir aktarildi
▶ 18:25:25 [ YÜKLE (XMODEM-CRC)] tıklandı
▶ 18:25:25   std_a_gauss_sum__...mem • 36 bayt • 1 paket
▶ 18:25:25 [OK] FPGA 'C' aldı (CRC modu).
▶ 18:25:25   paket 1/1 ACK
▶ 18:25:25   BAŞARILI • paket=1 retry=0 süre=0.04s

```

14.4 Zaman Damgalı Dosya İsimleri

Her derlemede yeni isimle dosya üretilir, eskileri korunur:

Eski (üstüne yazılır)	Yeni (her seferinde benzersiz)
test1.o	test1__20260601_182500.o
test1.mem	test1__20260601_182500.mem
firmware.mem	test1__20260601_182530.mem

Format: <stem>__YYYYMMDD_HHMMSS.<ext>. Bu sayede sunum sırasında farklı testlerin sonuçları karışmaz, log korelasyonu kolaylaşır. Eski dosyalar ☐ build butonu ile toplu silinebilir.

14.5 RV32I Disassembler

IDE'nin İnceleme sekmesinde, .mem dosyasındaki her 32-bit kelimeyi Python ile disassemble eder. Eğitsel amaç: öğrenci üretilen kodu doğrulayabilir.

```
def disassemble(w):
    op = w & 0x7F
    rd = (w >> 7) & 0x1F
    f3 = (w >> 12) & 0x7
    rs1 = (w >> 15) & 0x1F
    rs2 = (w >> 20) & 0x1F
    f7 = (w >> 25) & 0x7F
    # imm cikarimi formata gore
    if op == 0x33: return f"{mn:6} {_r(rd)}, {_r(rs1)}, {_r(rs2)}" # R
    if op == 0x13: return f"{mn:6} {_r(rd)}, {_r(rs1)}, {imm_i}" # I
    if op == 0x37: return f"lui    {_r(rd)}, 0x{imm_u >> 12:05X}" # U
    if op == 0x6F: return f"jal    {_r(rd)}, {imm_j:+}" # J
    # ...

# Ornek cikti:
# 0x00000000 lui    a0, 0x10000
# 0x00000004 addi   a1, zero, 0
# 0x00000008 sw     a1, 0(a0)
# 0x0000001C bne    t0, zero, -4  <- ileri/geri offset gorulebilir
```

15. Performans Metrikleri

15.1 Yükleme Süresi vs. Kod Boyutu

Test	Boyut (bayt)	Paket Sayısı	Süre (s)	Etkin Hız (B/s)
std_a (Gauss)	36	1	0.04	900
std_b (Sort)	≈ 100	1	0.05	2 000
std_c (Fib)	≈ 80	1	0.04	2 000
Sentetik 4 KB	4 096	32	0.42	9 750
Sentetik 32 KB	32 768	256	3.40	9 650

Maksimum teorik UART hızı: $115200 / 10 = 11520$ bayt/s. Bizim 9750 B/s ölçümümüz = **%85 verimlilik** (ACK paketleri ve CRC gecikmesi nedeniyle).

15.2 FPGA Kaynak Tüketimi

Kaynak	Kullanılan	Toplam	Yüzde
LUT (CLB)	~ 2200	8640	≈ 25%
Register (Flip-Flop)	~ 1500	6480	≈ 23%
BSRAM (16 Kbit)	16	26	61%
GPIO Pin	11	63	17%
DSP / PLL	0	—	—

BSRAM kullanımı 32 KB RAM (4 bayt-array × 4 BSRAM bloğu) + PicoRV32'nin kayıt dosyalarından kaynaklanmaktadır. Loader FSM + UART + CRC16 yaklaşık 250 LUT tüketir; bu "donanım FSM + basit veri akışı" yaklaşımının alan-verimli olduğunu doğrular.

15.3 Hata Toleransı (CRC Saldırı Testi)

XMODEM CRC-16 mekanizmasının etkinliğini kanıtlamak için host loader'da **kasıtlı bit-bozulma simülasyonu** uygulandı:

```
# build_packet sonuna eklenti (test amaçlı):
import random
if random.random() < 0.10:           # %10 paket bozulmasi
    payload = bytearray(payload)
    payload[0] ^= 0x01                # ilk bit ters cevir
```

Senaryo	Toplam Paket	Ortalama Retry	Süre Artışı	Veri Kaybı
Temiz hat (kontrol)	64	0	+ 0 %	0
%10 bozulma simülasyonu	64	6.4	+ 12 %	0
%30 bozulma simülasyonu	64	21	+ 38 %	0

Sonuç: Mimaride tercih edilen seri CRC-16/LFSR yaklaşımının UART'ın asenkron çerçeveleme zafiyetlerine karşı paket-düzeyinde **%100 koruma** sağladığı ampirik olarak kanıtlandı. Checksum yaklaşımına kıyasla simetrik bit hatalarında üstündür.

16. Sonuç ve Kaynakça

16.1 Genel Değerlendirme

PicoRV32 FPGA UART Loader projesi, RISC-V tabanlı bir SoC'nin tüm katmanlarını — assembler, linker, host yazılım, UART protokolü, CRC doğrulaması, RTL donanım, bellek hiyerarşisi, GPIO — özgün olarak tasarlayıp entegre ederek **uçtan uca** çalışan bir geliştirme platformu inşa etmiştir.

Tasarımın temelinde **mimari sözleşme** (architectural contract) felsefesi yatar: 5 ayrı katmanın (linker, CPU reset, loader, BRAM, adres decoder) aynı bellek haritası üzerinde anlaşması. Bu sözleşme değiştirildiğinde tüm katmanların eş zamanlı güncellenmesi gerekir; bu da gerçek hayattaki SoC tasarım süreçlerinin küçük ölçekli bir yansımasıdır.

16.2 Karşılaşılan Zorluklar ve Çözümler

Sorun	Belirti	Çözüm
Pass 1 yorum bug	Branch offset'ler kayık	strip_comment() ile # ; // sil
BRAM 262K FF	IF0008 hatası	4 bayt-array + dual-source mux
ABI register yok	Sessiz x0 fallback	psABI tablosu eklendi
Hex literal yok	0x3F → 0	parse_imm() ile strtol(0)
WDAC engelliyor	.exe çalışmıyor	Unblock-File / Add-MpPreference
PowerShell scripts	Execution policy	Set-ExecutionPolicy Bypass

16.3 Geleceğe Yönelik İyileştirmeler

- **Interrupt desteği:** PROGADDR_IRQ + timer/buton interrupt
- **SPI flash boot:** Bitstream sonrası kalıcı firmware
- **UART hızı:** 921600 baud (gürültü toleransı testi gerekir)
- **Çok-segment .mem:** @ adresi atlama destekli loader
- **RV32IM:** M extension (donanım çarpma/bölme)
- **Debug arayüzü:** JTAG veya UART-based GDB stub
- **Daha büyük BRAM:** External SDRAM controller

16.4 Kaynakça

- [1] D. A. Patterson & J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*, 1st ed., Morgan Kaufmann, 2017. ISBN 978-0-12-812275-4.
- [2] A. Waterman & K. Asanović (Eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, December 2019.
- [3] RISC-V International, *RISC-V ABIs Specification v1.0*, 2022. <https://github.com/riscv-non-isa/riscv-elf-psabi-doc>
- [4] C. Wolf, *PicoRV32 — A Size-Optimized RISC-V CPU*, <https://github.com/YosysHQ/picorv32>

- [5] W. W. Peterson & D. T. Brown, "Cyclic codes for error detection," *Proc. IRE*, vol. 49, no. 1, pp. 228-235, Jan. 1961.
- [6] Ward Christensen, *XMODEM Protocol*, 1977; Chuck Forsberg, *XMODEM/YMODEM Protocol Reference*, 1988.
- [7] IEEE Std 1364-2005, *IEEE Standard for Verilog Hardware Description Language*, Section 17 (System Tasks), IEEE Computer Society, 2005.
- [8] RISC-V International, *riscv-tests Repository*, <https://github.com/riscv-software-src/riscv-tests>
- [9] Massachusetts Institute of Technology, *6.004 Computation Structures*, Spring 2023 Lab 6: Recursion on RISC-V, <https://6004.mit.edu/>
- [10] Free Software Foundation, *GNU Assembler (gas) for RISC-V*, binutils source tree [gas/config/tc-riscv.c](https://sourceware.org/binutils/docs/as/)
- [11] Sipeed, *Tang Nano 9K User Manual*, <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-9K/Nano-9K.html>
- [12] Gowin Semiconductor, *GW1N Family of FPGA Products Data Sheet*, DS100E-1.4E, 2023.

Bu doküman BIL302 Sistem Programlama Proje 3 kapsamında hazırlanmıştır. Tüm modül kaynakları https://github.com/yasinyilmaaz/PicoRV_Assembler_Tasar-m- adresinde açık erişim olarak yayınlanmıştır.